INFORME TÉCNICO: *MOTOR DE BÚSQUEDA DE FUNCIONES SIMILARES VÍA AST*

Proyecto: Detección de similitud estructural en código Python | ***Asignatura:*** Lenguajes Formales y Autómatas — UCSE (2026)
SHARO GENNERO – CHRISTIAN MALDONADO

1. Objetivo y arquitectura

El trabajo desarrolla una herramienta de análisis estático capaz de identificar funciones estructuralmente similares dentro de un repositorio Python. En lugar de comparar texto plano, utiliza el Árbol de Sintaxis Abstracta (AST), que representa la estructura sintáctica del programa y permite reducir el efecto de diferencias superficiales, como el formato, los comentarios, los nombres de variables y los valores literales.

La solución se organiza en cinco módulos: **extracción** (extractor.py), que recopila las funciones del repositorio; **normalización** (normalizer.py), que transforma sus árboles a una representación comparable; **serialización** (serializer.py), que convierte los árboles en secuencias de tokens; **similitud** (similarity.py), que calcula la semejanza entre secuencias; y **búsqueda** (engine.py y find_similar.py), que indexa el corpus y permite realizar consultas mediante una interfaz CLI. Esta arquitectura modular facilita el mantenimiento y las pruebas.

El proyecto vincula los contenidos de la asignatura con el análisis sintáctico: utiliza la estructura del AST, construida a partir del análisis gramatical del lenguaje, para comparar fragmentos de código.

2. Decisiones de diseño

Extracción: El sistema recorre recursivamente los archivos Python, omitiendo directorios de entorno o caché, y recopila funciones convencionales, asíncronas, métodos y funciones anidadas. Registra el archivo, el nombre, el ámbito y la línea de definición para distinguir funciones homónimas. Los errores de sintaxis o codificación se notifican sin interrumpir el procesamiento del resto del repositorio.

Normalización: Se construye una tabla de símbolos local que permite identificar parámetros y variables asignadas, contemplando distintos contextos de asignación y excluyendo las declaraciones global y nonlocal. Los identificadores locales se reemplazan por nombres canónicos (VAR1, VAR2, etc.); los literales numéricos y de texto, por LIT; y el nombre de la función raíz y sus llamadas recursivas, por FUNC. Se conservan los nombres libres, los atributos y las constantes relevantes (None, True, False y Ellipsis). También se eliminan los *docstrings* y las anotaciones de tipos para reducir diferencias que no son centrales para la comparación.

Serialización: Se implementa un recorrido en profundidad y preorden (DFS) mediante ast.iter_child_nodes(), en lugar de ast.walk(), cuyo recorrido es en anchura. Los nodos estructurales emiten su tipo (If, Return, BinOp) y los que contienen información relevante, una combinación de tipo y valor (Name:VAR, Constant:LIT). Se omiten los contextos Load, Store y Del.

Por defecto, index_vars=False unifica los identificadores canónicos bajo VAR, lo que puede mejorar la tolerancia al reordenamiento de instrucciones independientes. Con --index-vars, se conservan los índices (VAR1, VAR2), preservando más información sobre la identidad de las variables, aunque ciertos reordenamientos pueden reducir la similitud.

3. Métricas y evaluación experimental

Se implementaron dos métricas:

- ***Jaccard sobre n-gramas:*** compara conjuntos de subsecuencias contiguas de tokens. Con n = 3, calcula la proporción entre los trigramas compartidos y el total de trigramas distintos ($|A \cap B| / |A \cup B|$). Es la métrica predeterminada.

• ***Levenshtein normalizada:*** calcula el costo mínimo de inserciones, eliminaciones y sustituciones de tokens, normalizado respecto de la longitud de la secuencia más larga. Produce una puntuación entre 0 y 1, donde los valores mayores indican más similitud.

La calibración (lab/calibrate.py) evaluó nueve pares: cuatro positivos, correspondientes a clones con renombrado, reordenamiento o modificaciones puntuales, y cinco negativos, con funciones de algoritmos diferentes.

Caso	Jaccard, índices activados	Jaccard, índices desactivados	Levenshtein, índices desactivados
C1: Renombrado de variables y literales	1,000	1,000	1,000
C2: Reordenamiento de instrucciones	0,451	0,867	0,846
C3: Clon con guarda adicional	0,645	0,667	0,774
C4: Clon con llamada de logging	0,692	0,692	0,846
Mínimo entre positivos	**0,451**	**0,667**	**0,774**
Máximo entre negativos	**0,163**	**0,220**	**0,655**
Margen de separación	**0,288**	**0,447**	**0,119**

Los resultados respaldan la elección de trigramas: los bigramas alcanzaron hasta 0,400 de similitud entre funciones distintas, mientras que los n-gramas de longitud cuatro penalizaron más algunas inserciones. En los pares evaluados, desactivar los índices elevó la similitud del caso de reordenamiento de 0,451 a 0,867 y amplió el margen entre positivos y negativos de 0,288 a 0,447.

Se establecieron umbrales de **0,50 para Jaccard** y **0,72 para Levenshtein**. Ambos clasificaron correctamente los nueve pares evaluados. Este resultado valida las decisiones para la muestra utilizada, pero no garantiza la misma precisión ante otros repositorios o casos.

4. Limitaciones y mejoras

La herramienta identifica similitud sintáctica (clones de Tipo 1, 2 y 3), no equivalencia funcional (clones de Tipo 4). Por ello, algoritmos equivalentes expresados mediante estructuras diferentes, como iteración y recursión, pueden no reconocerse como clones. Las funciones muy breves también pueden producir falsos positivos: en las pruebas, expresiones como return self.name y return self.age alcanzaron una similitud de Levenshtein de 0,833. Además, index_vars=False pierde información sobre la identidad de los operandos, por lo que expresiones como a + a y a + b pueden generar la misma representación.

Como mejoras futuras, se propone incorporar un filtro de longitud mínima para descartar funciones triviales, mejorar el tratamiento de variables capturadas por funciones anidadas, ampliar la calibración con repositorios reales y ponderar los nodos estructurales más informativos. Para repositorios grandes, podría añadirse una etapa de preselección de candidatos, por ejemplo, mediante *MinHash*, para reducir la cantidad de comparaciones del esquema todos contra todos, cuyo costo es cuadrático respecto de la cantidad de funciones.

En conclusión, el proyecto implementa un motor modular de búsqueda de similitud estructural que combina normalización del AST, serialización en preorden y métricas cuantitativas. La evaluación inicial permite justificar sus decisiones de diseño e identificar sus principales limitaciones.